# Real-World Applications of Stacks as a Data Structure

**Assignment:** Data Structures & System Implementation **Repository:** github.com/lk-chris/Weather-App

## 1. WHAT IS A STACK & WHY IS IT NECESSARY?

A **Stack** is a linear data structure that operates on the **Last-In, First-Out (LIFO)** principle: whatever was placed into the stack most recently is always the first element removed, just like a stack of plates or trays. Adding an element is called `push` (placed at the top), and removing an element is called `pop` (retrieved from the top). Both operations, along with inspecting the top element with `peek`, execute in instantaneous *O(1)* constant time.

**Why Stacks are Necessary in Computing:**

- **Execution Context & Function Calls (The Call Stack):** When programs run nested or recursive functions, the computer must remember where to return when each function finishes. A stack records each active function frame on top and pops it when the function completes, keeping memory execution orderly.
- **Reversible State Management (Undo / Redo & History):** Applications frequently require users to step backward through actions. By pushing each action or screen state onto a stack, the previous state is always preserved right beneath the top, making reversal a simple single-step pop.
- **Syntax Parsing & Delimiter Matching (Expression Validation):** Compilers and code linters check nested brackets like `{ [ ( ) ] }`. Opening symbols are pushed to the stack and popped when closing symbols match. Any leftover or mismatched symbol immediately catches syntax errors.
- **Algorithmic Backtracking (Depth-First Search):** When searching through mazes, decision trees, or network paths, a stack records the route explored so an algorithm can cleanly backtrack to the most recent decision point whenever it reaches a dead end.

## 2. REAL-WORLD APPLICATIONS OF STACKS

### 1. Browser Navigation History

Browsers push visited URLs onto a history stack; pressing the "Back" button pops the top page to reveal and restore the preceding one.

### 2. Text Editor Undo/Redo Engine

Every keystroke, paste, or deletion is pushed onto an undo stack. Pressing Ctrl+Z pops the latest operation to restore prior document state.

### 3. Runtime Call Stack & Recursion

Language runtimes allocate execution frames on a call stack to track function arguments, local variables, and return addresses safely.

### 4. Syntax Analysis & Expression Parsing

Compilers use stacks to convert infix arithmetic into postfix notation (Reverse Polish) and verify that code delimiters open and close properly.

## 3. SELECTED APPLICATION TO DEVELOP: WEATHER DASHBOARD & SEARCH HISTORY STACK

### Application Overview & Stack Architecture

The application is a **Weather Forecast Dashboard** featuring an interactive search bar whose historical query cache and navigation controls are governed by an internal **LIFO (Last-In, First-Out) Stack**. When users search for global cities, each location state is **pushed** onto the search history stack. Clicking the history clock icon opens a dropdown rendering past queries in strict LIFO order, keeping the most recent search at the top of the list, while integrated back and forward controls allow immediate stack-based view restoration.

- **LIFO History Dropdown:** Accessing the search history icon renders past queries in reverse chronological order from the top of the stack (`stack[top]` down to `stack[0]`), ensuring the most recent search is immediately accessible.
- **Push Operation (Search Query):** As new locations are queried, complete weather snapshots (coordinates, temperature, atmospheric data) are **pushed** onto the history stack in instantaneous *O(1)* time, clearing the forward traversal stack.
- **Pop Operation (Back Navigation):** Clicking the Back button **pops** the active city off the top of the stack and pushes it to the forward stack, seamlessly restoring the previous city to the view in *O(1)* time with zero redundant network queries.